ENGINEERING SPECIFICATION DOCUMENT

DevFolio MERN

System Architecture, Technical Flowcharts, Core Working Mechanics & Production Engineering Specification

ARCHITECTURE PATTERN

Decoupled Client-Server MERN Architecture with RESTful APIs & Stateless JWT Session Control

FRONTEND CORE

React 19, Vite, React Router 7, Tailwind CSS v4, DaisyUI v5, Context API, Client Canvas Engine

BACKEND & STORAGE

Node.js, Express 5 REST API, MongoDB Atlas, Mongoose ODM, Bcrypt Password Hashing

SYSTEM PERFORMANCE

Client-side 256px Downscaling, O(1) Atomic Document Persistence, Zero Mobile Viewport Sway

Repository: github.com/ridhi431/mernprac

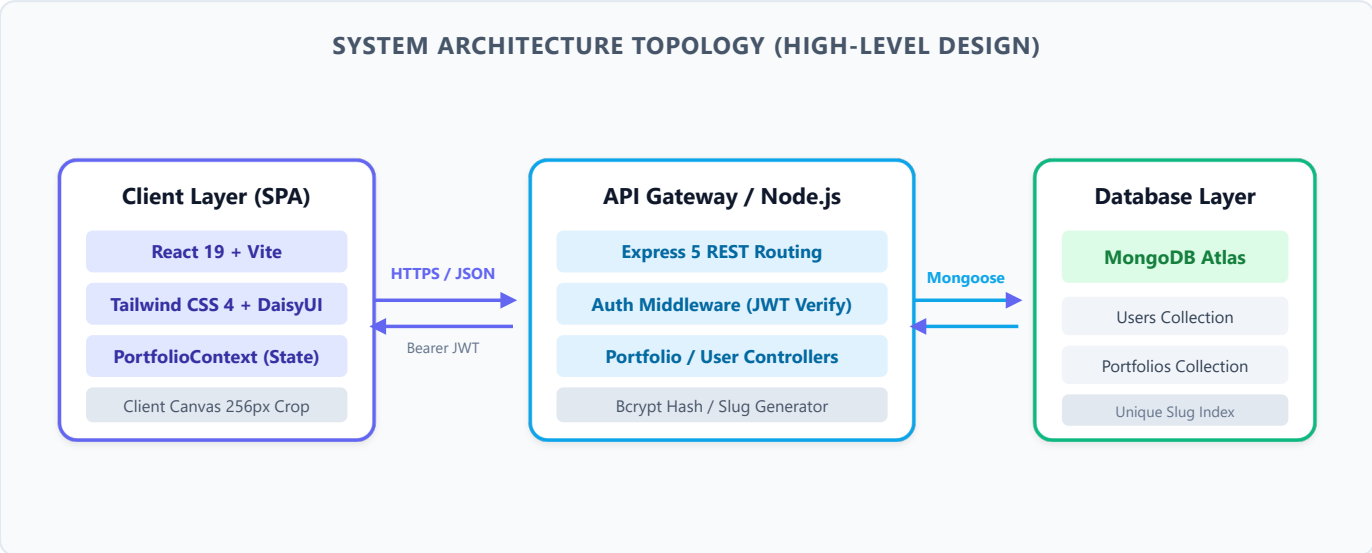
Live Demo: mernprac.vercel.app

License: MIT Open Source

Version: Production 1.0 (2026)

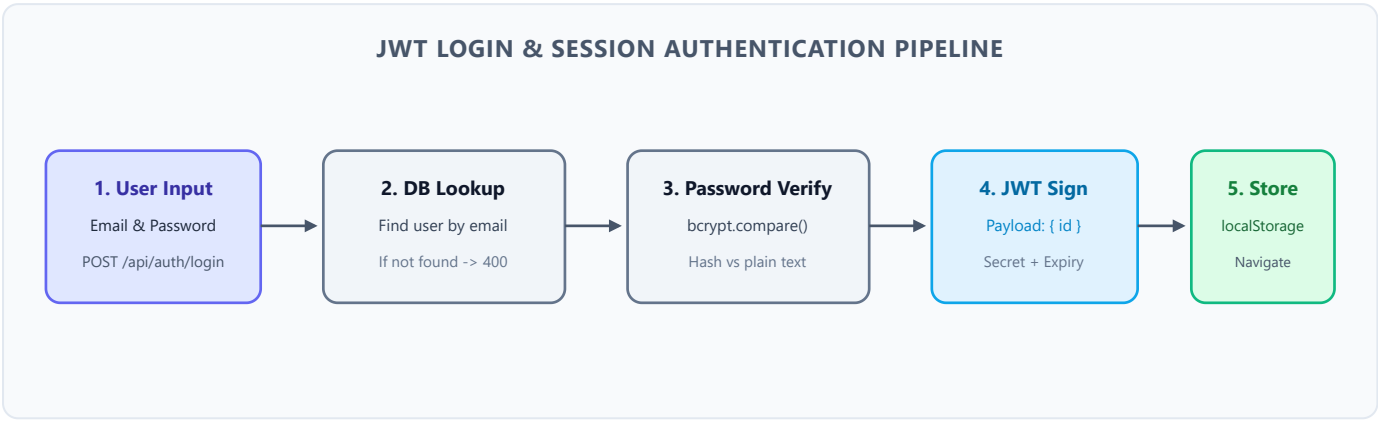
1. System Overview & High-Level Architecture

DevFolio is an end-to-end full-stack portfolio builder built using the modern **MERN** stack (MongoDB, Express.js, React 19, Node.js). It empowers developers to build, customize, edit, preview, and host professional portfolio websites in under two minutes with personalized public URLs.

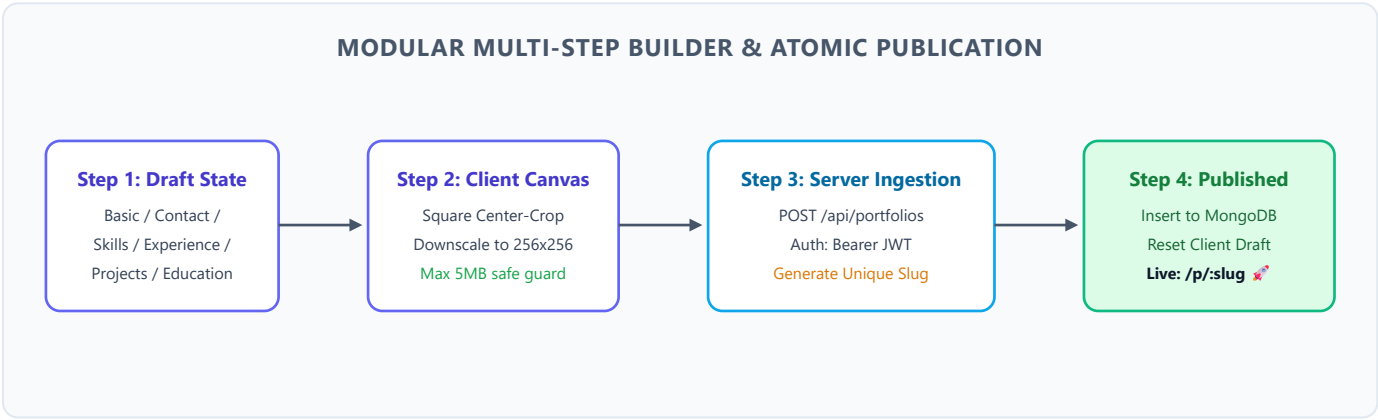


2. Technical Flowcharts & Execution Sequences

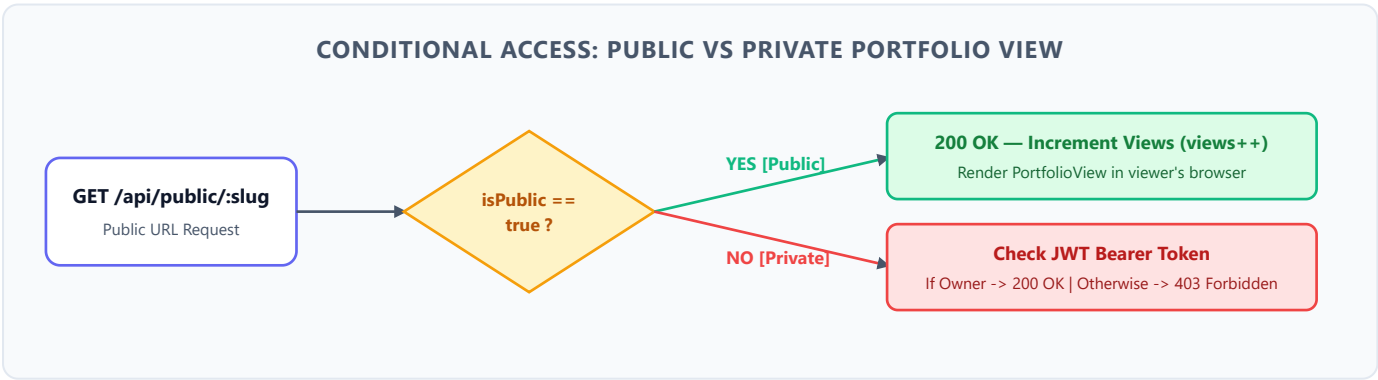
Flowchart 2.1: Authentication & Session Issuance



Flowchart 2.2: Portfolio Creation & Publishing Pipeline



Flowchart 2.3: Public Slug Resolution & Security Access Gate



3. Database Schema Design & Data Modeling

DevFolio utilizes **MongoDB** with **Mongoose** for document modeling. The data model is optimized for high-read throughput on public portfolio endpoints and fast nested updates on draft forms.

3.1 User Schema (`models/User.js`)

Field	Type	Constraints	Description & Purpose
`name`	String	Required, Trimmed	Full name of the user/creator
`email`	String	Required, Unique, Lowercase	Primary login identifier & unique account index
`password`	String	Required, Min length 6	One-way hashed password via Bcrypt (salt rounds = 10)
`photo`	String	Optional Base64 / URL	Profile avatar image (canvas downscaled to 256x256)
`bio`	String	Optional, Max 500 chars	Short description or developer summary
`timestamps`	Date	createdAt, updatedAt	Automatic audit timestamps managed by Mongoose

3.2 Portfolio Schema (`models/Portfolio.js`)

Field	Type	Index & Constraints	Description
`user`	ObjectId	Ref: 'User', Required, Indexed	Foreign key linking portfolio to its creator
`slug`	String	Unique, Required, Indexed	Human-readable URL slug (e.g. `/p/alex-fullstack-dev-7a8f`)
`theme`	String	Default: 'light'	DaisyUI theme identifier (17+ themes supported)
`isPublic`	Boolean	Default: true	Privacy toggle controlling global visibility
`views`	Number	Default: 0	Aggregated page view counter, incremented per public visit
`basic`	Object	Subdocument	`{ name, title, bio, photo }`
`contact`	Object	Subdocument	`{ email, github, linkedin }`
`skills`	[String]	Array of Strings	List of technical skills and tools
`projects`	[Object]	Array of Subdocuments	`[{ title, description, link }]`
`experience`	[Object]	Array of Subdocuments	`[{ company, role, duration, description }]`
`education`	[Object]	Array of Subdocuments	`[{ institute, degree, year }]`

> **Design Tradeoff Rationale:** We embedded `projects` and `experience` directly inside the Portfolio document instead of creating separate collections with relational references. Since a portfolio document is almost always read as a single atomic unit, embedding eliminates costly `$lookup` joins, cuts database roundtrips to 1, and enables fast caching.

4. Core Engineering & System Implementations

4.1 Client-Side Canvas Image Downscaling Engine

Rather than uploading raw multi-megabyte photos to the Node server, DevFolio crops and compresses the photo inside an HTML5 Canvas element in the browser before network transmission.

```
const resizeImage = (file, size = 256) =>
  new Promise((resolve, reject) => {
    const reader = new FileReader();
    reader.onload = () => {
      const img = new Image();
      img.onload = () => {
        const canvas = document.createElement('canvas');
        canvas.width = size;
        canvas.height = size;
        const min = Math.min(img.width, img.height);
        const sx = (img.width - min) / 2;
        const sy = (img.height - min) / 2;
        const ctx = canvas.getContext('2d');
        ctx.drawImage(img, sx, sy, min, min, 0, 0, size, size);
        resolve(canvas.toDataURL('image/jpeg', 0.8)); // 80% JPEG quality
      };
      img.src = reader.result;
    };
    reader.readAsDataURL(file);
  });
```

4.2 Mobile Responsiveness & Viewport Stabilization

- **Zero Horizontal Sway:** Enforced `overflow-x: hidden` and `max-width: 100vw` on `html, body, #root` to prevent mobile wobble from floating animated blobs.
- **Universal Drawer Architecture:** Synchronized DaisyUI mobile drawer across both `Mainlayout` and `Dashboardlayout` with auto-close navigation on link tap.
- **Touch-Device 3D Tilt Suppression:** Used `window.matchMedia('(hover: none)')` to prevent 3D perspective cards from freezing at awkward angles during touch swipes.

4.3 Complete REST API Reference

Method	Endpoint	Auth Level	Description
`POST`	`/api/auth/signup`	Public	Registers user, hashes password, returns JWT
`POST`	`/api/auth/login`	Public	Validates credentials, returns JWT
`GET`	`/api/users/me`	Private (Bearer)	Fetches logged-in user profile & photo
`PUT`	`/api/users/profile`	Private (Bearer)	Updates name, bio, and profile photo
`GET`	`/api/portfolios`	Private (Bearer)	Retrieves all portfolios belonging to user
`POST`	`/api/portfolios`	Private (Bearer)	Creates & publishes a new portfolio
`GET`	`/api/portfolios/:id`	Private (Owner)	Retrieves single portfolio by MongoDB ID for edit

Method	Endpoint	Auth Level	Description
`PUT`	`/api/portfolios/:id`	Private (Owner)	Updates existing portfolio
`DELETE`	`/api/portfolios/:id`	Private (Owner)	Deletes portfolio permanently
`GET`	`/api/portfolios/public/:slug`	Public / Conditional	Resolves public portfolio view & increments views

5. Security, Fault Tolerance & Scalability Architecture

5.1 Password Hashing & Bcrypt Work Factor

Passwords are never stored in plain text. We utilize `bcryptjs` with 10 salt rounds (\(2^{10} = 1024\) iterations). This adds a computational cost of ~70ms on modern server hardware—completely imperceptible to a human user logging in, but computationally intractable for brute-force attacks or rainbow table lookup tables.

5.2 NoSQL Injection Mitigation

In unvalidated MongoDB drivers, passing raw JSON objects containing query operators (e.g. `{"$gt": ""}`) can bypass password checks. In DevFolio, input values are strictly destructured and validated through Mongoose Schemas with explicit String casting, neutralizing injection vectors.

5.3 Concurrency & Unique Slug Collision Handling

DevFolio enforces uniqueness on two layers:

- **Database Layer:** A B-Tree unique index on the `slug` field prevents duplicate inserts at the storage level.
- **Application Layer:** Slugs combine sanitized user names with a collision-resistant random entropy suffix. If two concurrent writes collide, MongoDB returns error code `11000`, which is caught by the controller to automatically generate a fresh suffix and retry insertion.

5.4 Scalability Architecture (10M Daily Views Blueprint)

- **Edge CDN Caching:** Public portfolio pages are served via Cloudflare or Vercel Edge with Cache-Control headers, serving 95% of reads directly from edge points of presence.
- **Redis Caching Layer:** Published portfolio JSON documents are cached in Redis with a 10-minute TTL.
- **Decoupled View Count Buffering:** Instead of writing to MongoDB on every page view, view counts are incremented via an atomic Redis counter (`INCR portfolio:views:{id}`) and flushed asynchronously in batches every 60 seconds.
- **Stateless Compute:** The Express application maintains no local state or session data, enabling instant horizontal autoscaling across container clusters.